

2026 年编译系统挑战赛项目文档

动态算子图编译 与并行调度

功能、性能、创新性与测试分析报告



文档数据冻结：2026 年 8 月 5 日

对应仓库版本：7e07d6a

摘要

本项目面向“动态算子图编译与并行调度”赛题，通过 LLVM Pass 从优化后的 IR 中识别算子与循环结构，生成异步版本，并插入通用运行时接口。当前默认路径采用同步 `cholesky`、TRSM range、阶段同步、二维分组 MADD range、阶段同步的调度方式；对无依赖 phase 使用持久 epoch session 与静态领取，避免细粒度任务反复经过共享队列。

项目的主要工程创新包括：以版本化函数 annotation 为 IR 语义契约，在地址表达被 GEP/PHI 折叠后恢复块坐标；以一维与二维 range task 在编译器与通用运行时之间传递迭代空间；以测量证据选择“两阶段屏障”与持久无锁 phase，而非维护成本更高的逐任务依赖图；以及贯通正确性验证、优化后 IR 断言、运行时 profile 与逐用例评分的数据闭环。

最接近官方平台的有效结果来自 40 核 HiSilicon AArch64、openEuler 22.03、BiSheng/Clang 15.0.4。

关键词：LLVM Pass；任务图；分块 Cholesky；并行运行时；任务粒度；AArch64

目录

摘要	I
第 1 章 项目背景与目标	1
1.1 赛题任务	1
1.2 设计目标与边界	1
1.3 提交物组成	2
第 2 章 功能与系统设计	3
2.1 总体流程	3
2.2 LLVM Pass 功能	3
2.2.1 IR 版本化与运行时分派	3
2.2.2 算子 outline 与 ABI 保留	3
2.2.3 版本化语义 annotation	4
2.2.4 默认调度 lowering	4
2.2.5 TRSM range 与二维 MADD range	4
2.3 运行时功能	5
2.4 功能清单	5
第 3 章 创新性说明	7
3.1 创新点一：从低层 IR 恢复算子图语义	7
3.2 创新点二：编译器生成一维/二维语义任务，运行时只管理区间	7
3.3 创新点三：用证据选择同步结构	7
3.4 创新点四：可复现的性能证据链	7
第 4 章 测试方法与正确性	8
4.1 有效性能平台	8
4.2 评分口径	8
4.3 正确性与结构性检查	9
4.4 内存受限下的分段测量	9
第 5 章 性能结果与分析	10
5.1 全量公开用例结果	10

5.2	按块大小分桶	10
5.3	平台粒度重标定: Round 14 到 Round 15	11
5.4	Round 16: 线程扩展与执行归因	11
5.5	噪声与结论边界	12
第 6 章	优化迭代: 瓶颈定位与方案收敛	13
6.1	Git 历史中的主线	13
6.2	瓶颈一: 细粒度 DAG 的元数据和共享队列成本	13
6.3	瓶颈二: 任务粒度与目标平台不匹配	14
6.4	瓶颈三: 小 tile 的跨任务访存缺少复用	14
6.5	瓶颈四: 每 panel 的 TRSM 固定提交延迟	15
6.6	困难、负结果与路线收敛	15
6.7	当前尚未解决的瓶颈	16
第 7 章	后续路线与风险	17
7.1	优先级路线	17
7.2	主要风险与缓解	17
第 8 章	复现、构建与证据索引	18
8.1	构建与 smoke test	18
8.2	Round 28 当前基线同口径复现	18
8.3	关键证据文件	18
8.4	可调参数速查	20
第 9 章	总结	21

插图

2.1	编译与执行链路	3
2.2	当前默认的 panel 内两阶段调度	4
5.1	Round 28 当前主线按块大小分桶的几何平均加速比	10
5.2	Round 16: 不同块大小的线程扩展曲线 ($n = 1152$)	12

表格

1.1	项目目标与竞赛边界	1
2.1	主要功能与验证方式	5
4.1	Round 28 当前主线 AArch64 测试环境	8
5.1	AArch64 全量结果摘要	10
5.2	分桶统计与解释	11
5.3	Round 15 配对分组对照	11
6.1	关键提交与优化决策	13
6.2	工程困难与处理原则	15
7.1	项目风险	17
8.1	报告引用的仓库证据	18
8.2	主要运行时开关	20

第 1 章 项目背景与目标

1.1 赛题任务

赛题要求参赛者在 BiSheng/LLVM 编译链中增加优化 Pass，分析动态算子图中的依赖，生成可执行程序并在保证数值正确的前提下提高并行性能。目标程序是分块 Cholesky 分解，输入为双精度对称正定矩阵；矩阵规模 n 与块大小 b 在运行时给定。核心算子为：

- **cholesky**：分解当前对角块；
- **trsm**：利用已分解的对角块更新 panel 中的块；
- **madd**：更新 trailing matrix 中的输出块。

若将块坐标记为 (r, c) ，第 p 个 panel 的核心依赖为

$$\begin{aligned} C(p, p) &\rightarrow T(r, p), \\ T(r, p), T(c, p) &\rightarrow M(r, c, p), \\ M(p+1, p+1, p) &\rightarrow C(p+1, p+1). \end{aligned}$$

其中同一阶段内的大量 **trsm** 或 **madd** 互相独立，构成主要并行来源。

1.2 设计目标与边界

表 1.1: 项目目标与竞赛边界

维度	项目目标	明确边界
功能	从 IR 识别算子、构建并行执行路径、输出正确矩阵	不修改输入输出格式与 verifier
编译器	保留原函数并生成异步版本，自动插入 runtime 调用	不把整个算法替换为手写并行 Cholesky
算子	任务函数直接调用官方 ABI	不修改、重定义或绕过官方算子
运行时	提供线程池、队列、range task、依赖实验和 profile	不包含针对某个测试规模的预计算结果
性能	按逐用例几何平均评价，兼顾小、中、大块	不以 I/O、校验器或计时漏洞制造加速

1.3 提交物组成

提交工程由三部分组成：`submission/pass/dag_pass.cpp` 实现编译期识别与改写；`submission/runtime/dag_runtime.cpp` 实现通用并行执行；`submission/manifest.json` 声明 Pass 插件、运行时静态库和链接参数。构建产物为 `libcontestant_pass.so` 与 `libcontestant_runtime.a`，由评测机在目标架构上重新编译，提交包不携带本机二进制。

第 2 章 功能与系统设计

2.1 总体流程

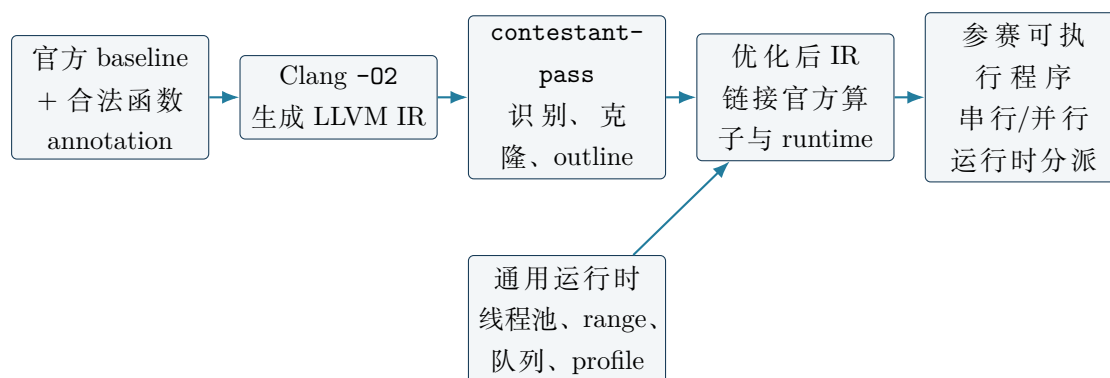


图 2.1: 编译与执行链路

2.2 LLVM Pass 功能

Pass 是 LLVM New Pass Manager 的 `FunctionPass`，只处理签名匹配的 `block_cholesky`。核心步骤如下。

2.2.1 IR 版本化与运行时分派

Pass 保留原始函数作为串行路径，克隆出 `compiler2026_async_impl`，并在入口插入 `compiler2026_runtime_should_async(n,b)`。这样，小规模或单块用例不会无条件承担线程池和同步开销；较大用例进入并行路径。运行时默认异步块阈值为 $b \geq 8$ ，最小块数为 2，同时结合 n 、硬件线程数和 `participant cap` 选择实际参与者。

2.2.2 算子 outline 与 ABI 保留

Pass 为算子生成任务 `context` 和内部任务函数。任务函数加载参数后直接执行 `@trsm` 或 `@madd`，原始 `call site` 则替换为运行时 `submit`。该设计保证：

- 官方算子实现仍是唯一数值计算实现；
- 运行时只认识函数指针、`context`、`range` 和可选整数依赖 `key`；
- 优化逻辑集中在 IR 变换与调度，不污染算法源码和官方 ABI。

2.2.3 版本化语义 annotation

提交版 `block_cholesky.cpp` 仅增加 `compiler2026.graph.block_cholesky.tile_dag.v1` 函数 annotation。它声明输出矩阵 L 的 row-major tile-DAG 语义，不携带规模、线程数或预计计算结果。默认路径据此安全地将内层 `madd` 循环提升为 range submit；实验性跨 panel 路径还能在 GEP 被优化为 PHI/指针差表达式时，从 L 基址恢复块坐标。

2.2.4 默认调度 lowering

当前默认路径不是完整跨 panel DAG，而是经实测选择的两阶段调度：

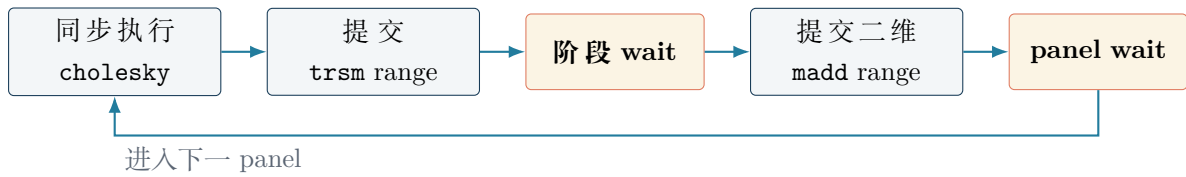


图 2.2: 当前默认的 panel 内两阶段调度

同一 panel 的 `trsm` 只依赖已同步完成的对角分解；第一次 wait 后，每个 `madd` 写不同输出块，因此第二阶段可以使用无依赖 range submit。相比逐 `madd` 建立两个输入依赖，阶段屏障牺牲少量 TRSM/MADD 重叠，却消除了单提交线程的哈希查询、DAG 节点与边构造成本。该选择来自实测否定，不是对依赖关系的忽略。

2.2.5 TRSM range 与二维 MADD range

Pass 把每个 panel 的 TRSM 行循环整体 outline 为一次 range submit。每个逻辑 index 仍对应一行官方 `trsm` 调用，但提交线程不再为每行重复分配 context 和执行 submit：

Listing 2.1: TRSM range 的语义化伪代码

```

void trsm_range(Context *ctx, int begin, int end) {
    for (int row = begin; row < end; ++row) {
        trsm(block(row), ctx->diag, block(row), ctx->b, ctx->n);
    }
}

```

MADD 阶段进一步把三角形 (row, col) 迭代域划分为二维 group。一个逻辑任务在同一执行核上连续处理一组输出块，使多个官方 `madd` 调用复用仍在 cache 中的 A/B tile：

Listing 2.2: 二维 MADD range 的语义化伪代码

```

void madd_tile_range(Context *ctx, int begin, int end) {
    for (int group = begin; group < end; ++group)
        for (auto [row, col] : decode_triangular_group(group))
            madd(A(row), B(col), C(row,col), ctx->b, ctx->n);
}

```

运行时只返回 group width 和 range 切分，不解释 (row, col) 的算子含义。默认宽度综合矩阵 block 数与 tile cache footprint：小 tile 使用 2-8 的分组宽度， $b \geq 28$ 时退化为 1。若 IR 形态不满足二维改写条件，Pass 保守回退到一维 MADD range；其默认目标工作量仍为 200000 FLOP。两条任务函数都直接调用官方 ABI。

2.3 运行时功能

运行时使用 `thread_local` 的调度器实例，核心能力包括：

- **可复用线程池**：按进程可能需要的最大 worker 数预热，后续只增不减；每次调用仅激活需要的参与者，多余 worker 停泊，避免跨 case 重建；
- **主线程参与**：wait 期间主线程执行 ready task，使“线程数”近似等于 main + workers；
- **arena context**：按调用批量分配 task context，避免逐任务 malloc/free；
- **持久无锁 phase**：对 $b < 128$ 的无依赖 phase，以 epoch 发布不可变任务表，worker 静态领取并写入 cache-line 对齐的完成标志，绕开逐任务队列锁；
- **暂存与批量出队**：成熟队列回退路径按参与者数积累足够宽度，并根据 ready width 自适应取批；
- **通用 range**：接收函数、context 与整数区间，支持默认粒度或显式 grain；
- **局部性分组**：只根据 n, b 返回二维 range group width，具体坐标映射仍由 Pass 掌握；
- **实验性 DAG**：保留 2/3 依赖 submit、priority、key wait 和 live-window，供跨 panel 路径验证，默认关闭；
- **可观测性**：可输出任务数、批量大小、ready width、queue/exec/idle 时间、DAG 节点/边、wait 压力以及按任务名聚合的统计。

2.4 功能清单

表 2.1: 主要功能与验证方式

功能	实现方式	默认状态	验证证据
串行/异步版本化	克隆函数并由 runtime predicate 分派	开启	优化后 IR call-site 断言
TRSM range	outline 行循环，每 panel 一次 range submit	开启	IR 中保留 @trsm
二维 MADD range	三角域按 (row, col) group 任务化	开启	IR 中保留 @madd
两阶段同步	TRSM 末尾与 MADD nest 末尾插入 wait	开启	ir_wait_calls=2
持久无锁 phase	epoch 发布、静态领取、逐 worker 完成标志	$b < 128$ 开启	Round 17 全量对照
线程池复用与预热	load-time 预热、调用间只增不减	开启	首个异步 case 对照

功能	实现方式	默认状态	验证证据
自适应粒度与分组	FLOP 预算 + cache/group width 模型	开启	Round 15/19 扫描
参与者选择	fast phase 按 n 阶梯限流, 队列路径按 b 限制	开启	Round 17–25 扫描
完整跨 panel DAG	块 key、三依赖 submit、单 outer wait	实验, 关闭	verifier smoke + 对照 CSV
关键路径优先级	Pass 生成 0–3 rank	实验, 关闭	本机重复实验未胜出
worker pinning	读取 Linux affinity mask 并绑定	实验, 关闭	profile smoke
profile CSV	stderr 计数解析并并入 benchmark CSV	按需开启	多份 profile CSV

第 3 章 创新性说明

3.1 创新点一：从低层 IR 恢复算子图语义

普通函数调用只暴露指针，经过 `-O2` 后，块地址可能表现为嵌套 GEP、PHI 或已折叠的指针算术。项目将 LoopInfo、调用参数模式和版本化 annotation 结合：默认路径识别稳定循环形态，实验路径以 L 为语义基址计算 element offset，再恢复 (row, col) 与线性 block key。其价值在于把“源代码里显然的 tile 坐标”带回优化后的 IR，同时只增加一条允许的函数级标注。

3.2 创新点二：编译器生成一维/二维语义任务，运行时只管理区间

range task 没有把 `trsm/madd` 逻辑搬入 runtime。编译器生成的任务函数掌握参数推进和三角域坐标映射，runtime 仅决定区间如何切块、二维 group 多宽。这种职责拆分同时满足三个要求：保留官方算子 ABI、让粒度与局部性按目标平台调整、保持调度器可复用于其他循环任务。

3.3 创新点三：用证据选择同步结构

完整依赖图在理论上暴露更多重叠，但逐任务依赖维护可能比重叠收益更昂贵。项目通过 profile 发现单提交线程构图时，worker 在“等待新节点”和“争抢共享队列”之间振荡。于是默认路径将可证明安全的阶段边压缩为两个 wait，并进一步为无依赖 phase 建立持久 epoch session；依赖 submit 仍保留为实验通道。该方法体现的是 **以任务图成本模型选择表示**：不是 DAG 越细越好，而是应让依赖表达的成本与可获得的关键路径收益匹配。

3.4 创新点四：可复现的性能证据链

项目将以下证据组合成闭环：

1. verifier 过滤所有数值错误；
2. 优化后 IR 统计 submit、wait 和官方算子调用，防止“跑得快但没改到”或越界改写；
3. profile 将队列宽度、任务执行、空闲与依赖状态写入 CSV；
4. 逐用例脚本按评测公式计算等权几何平均；
5. 当总差异落在噪声带内时，按“机制上会被改动影响/不会被影响”做配对分组对照。

上述创新是相对于本赛题 baseline 的工程与编译优化创新，不主张其为通用任务运行时领域的首次提出。

第 4 章 测试方法与正确性

4.1 有效性能平台

表 4.1: Round 28 当前主线 AArch64 测试环境

项目	配置
处理器	HiSilicon AArch64, 40 核, 单 NUMA, 2.9 GHz 定频, L3 32 MiB
操作系统	openEuler 22.03 LTS, Linux 5.10
工具链	BiSheng Enterprise 3.2.0.1.B004, Clang/LLVM 15.0.4
内存约束	容器 cgroup 硬上限 2 GiB
用例	官方 150 个公开用例
重复方式	PASSES=3, REPEAT=1, 每个 case 取三次整进程运行的最小值
正确性	串行参考与参赛输出均运行官方 verifier

该平台与官方环境具有相同 ISA、OS 家族和 BiSheng/LLVM 主版本, 因此作为当前最有效的性能证据。40 物理核 x86_64 Xeon 结果只用于同机调试和机制归因, 不能与 AArch64 绝对时间混用。

4.2 评分口径

对第 i 个用例, 令串行时间为 $T_{0,i}$, 参赛程序时间为 T_i :

$$\begin{aligned}s_i &= \frac{T_{0,i}}{T_i}, \\ S_{geo} &= \exp\left(\frac{1}{N} \sum_{i=1}^N \ln s_i\right), \\ P &= 100 \frac{S_{geo}}{m_{ideal}}, \\ Score &= 0.4F + 0.6P.\end{aligned}$$

在功能分 $F = 100$ 、 $m_{ideal} = 32$ 时, $Score = 40 + 1.875S_{geo}$ 。逐用例等权意味着 $n = 128$ 与 $n = 2048$ 权重相同; 总时间比实际按计算量偏向大用例, 因此只能作为吞吐参考。

4.3 正确性与结构性检查

Round 28 当前主线全量结果为 **150/150 verifier PASS**。此外，逐用例测试在优化后 IR 中断言：存在 submit；存在两处 runtime wait；Pass 生成的任务函数中仍有官方 @trsm 与 @madd 调用。归档记录的默认路径计数为：

```
ir_submits=2 (plain=0, range=2, deps=0), ir_wait_calls=2,  
ir_trsm_calls=3, ir_madd_calls=4.
```

这里的计数是静态 call site 数，不是运行时任务总数。它用于证明变换形态和 ABI 保留，不能代替数值 verifier。

4.4 内存受限下的分段测量

150 个 case 的输入和输出同时驻留约 2.46 GB，超过目标容器 2 GiB 限制。项目按声明尺寸把 spec 切为不超过 600 MB 驻留数据的 5 段，分别测量后恢复全局 case index 再统一评分。计时单位是单次 block_cholesky 调用；线程池预热已移出计时区，因而分段不会改变单 case 计时定义。该方法避免 OOM，同时保留评测口径。

第 5 章 性能结果与分析

5.1 全量公开用例结果

表 5.1: AArch64 全量结果摘要

版本	PASS	几何平均	总时间比	折算总分
Round 15: AArch64 粒度重标定	150/150	4.370×	7.782×	48.19
Round 17: 持久无锁 phase	150/150	6.123×	9.617×	51.48
Round 20: 二维 MADD tiling	150/150	7.006×	11.257×	53.14
Round 27: TRSM range	150/150	7.386×	11.751×	53.85
Round 28: 当前 HEAD 校准	150/150	7.386502×	11.660466×	53.85

Round 28 的 150 个 speedup 中位数为 8.229×，最小值 0.891×，最大值 39.645×；11 个用例低于 1×，其中 6 个低于 0.95×。当前 HEAD 的代码与被测提交一致；Round 28 只在合并后重新建立当前基线，没有把历史分支成绩直接借给主线。

5.2 按块大小分桶

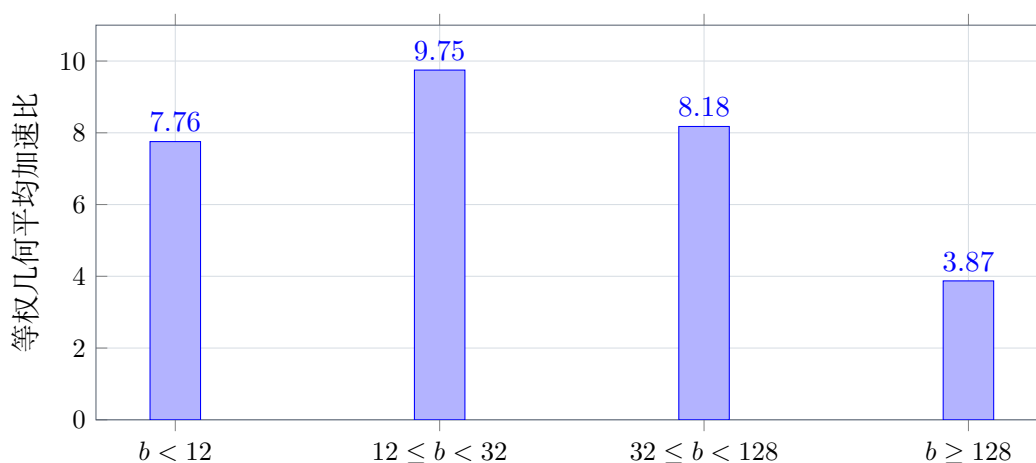


图 5.1: Round 28 当前主线按块大小分桶的几何平均加速比

表 5.2: 分桶统计与解释

块大小	用例数	几何平均	主要解释
$b < 12$	22	$7.755\times$	无锁 phase、二维复用与 TRSM range 消除了大部分细粒度固定成本
$12 \leq b < 32$	39	$9.748\times$	当前收益最高；仍受每 panel 固定延迟和必需内存流量影响
$32 \leq b < 128$	61	$8.176\times$	用例数最多，是继续提升全量成绩的主要覆盖区
$b \geq 128$	28	$3.872\times$	块数少，已达到相位结构上限约 94%，不再优先投入

5.3 平台粒度重标定：Round 14 到 Round 15

range task 的目标预算从 Xeon 上标定的 50000 FLOP 改为 AArch64 上的 200000 FLOP。全量几何平均从 $4.253\times$ 提高到 $4.370\times$ ，增幅 2.7%，小于该主机 5.7–9.6% 的运行间噪声，单看总数不足以采纳。项目按“两个预算是否会改变 chunk 长度”分组：

表 5.3: Round 15 配对分组对照

分组	用例数	50000 FLOP	200000 FLOP	变化
受影响 ($b \leq 36$)	80	$3.383\times$	$3.553\times$	+5.0%
对照 ($b \geq 40$)	70	$5.524\times$	$5.535\times$	+0.2%

对照组仅 +0.2%，说明主机整体漂移很小；受影响组 +5.0%，且 $b = 18, 24, 28$ 等组同向改善，因此该改动有机制一致的证据。这个案例同时说明：以 FLOP 表示“约若干微秒的任务”仍是平台相关代理量，不能从 x86 直接移植到 AArch64。

5.4 Round 16: 线程扩展与执行归因

Round 16 在同一 AArch64 平台固定 $n = 1152$ ，扫描 8/16/24/32/40 个参与者，分别观察小、中、大块。该实验用于定位瓶颈，不是新的 150 case 全量成绩。

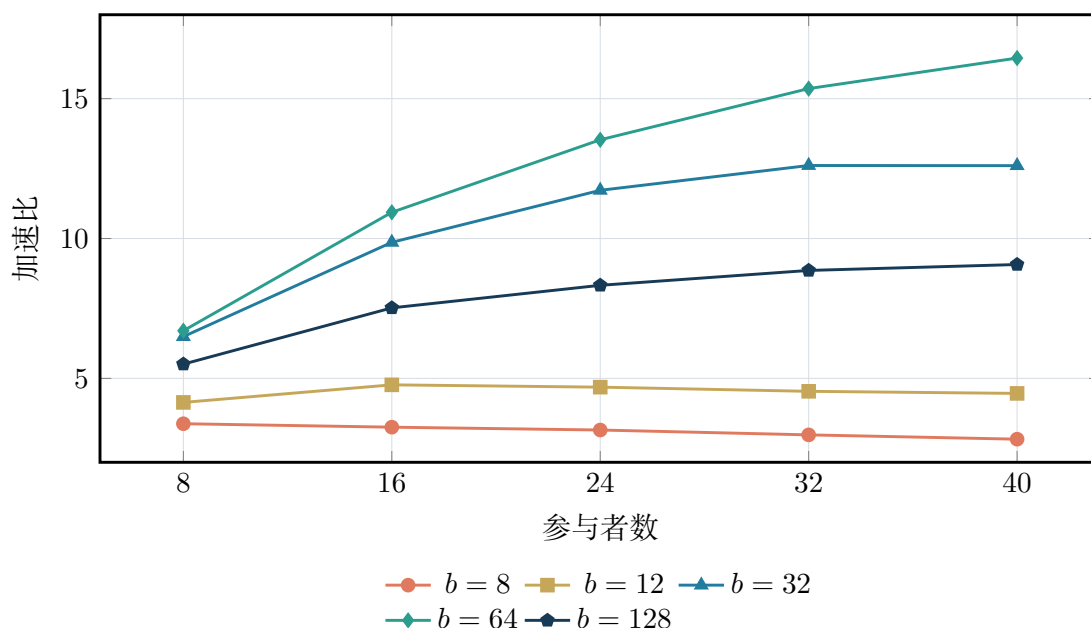


图 5.2: Round 16: 不同块大小的线程扩展曲线 ($n = 1152$)

$b = 8$ 从 8 参与者的 $3.379\times$ 下降到 40 参与者的 $2.827\times$ ，累计 worker idle 从 122.274 ms 增至 1504.563 ms； $b = 12$ 在 16 参与者达到峰值 $4.769\times$ ； $b = 32$ 在 32 参与者后基本饱和。对照中， $b = 64$ 从 $6.703\times$ 持续扩展到 $16.451\times$ ， $b = 128$ 从 $5.510\times$ 扩展到 $9.070\times$ 。这表明小块问题不是“任务图没有足够并行度”，而是增加参与者后执行效率下降；当时的候选原因是内存带宽/缓存复用不足或共享队列竞争。后续 Round 17 用持久无锁 phase 消除了逐 task 队列交接，Round 20 用二维 MADD 分组增加 A/B tile 复用；二维后 $b = 8$ 从 32 到 40 参与者仅由 $13.45\times$ 增至 $13.55\times$ ，最终确认该桶已进入带宽平台。

5.5 噪声与结论边界

性能数字的使用边界

同一配置的全量运行可出现约 5.7% 差异， $b = 8$ 还存在约 $\pm 10\%$ 的代码布局敏感性。因此 3% 以内的孤立差异不应宣称为优化收益；应采用重复测量、逐 case 配对、机制分组和不受影响对照组。AArch64 与 Xeon 之间只比较趋势，不能比较绝对时间或直接合并几何平均。

第 6 章 优化迭代：瓶颈定位与方案收敛

本项目的优化路径不是先确定一个复杂调度架构再一次实现，而是让每轮 profile、逐用例结果和负实验决定下一步。Git 历史完整保留了这种“发现问题—建立假设—做机制对照—采纳或回退—重新全量验证”的过程。

6.1 Git 历史中的主线

表 6.1: 关键提交与优化决策

阶段	提交	当时暴露的问题	决策与结果
Round 10	4a4a90d de41d5c	participant cap 掩盖真实调度缺陷；单提交线程构图时 worker 饿死	先否定 “ $2.4\times$ 是结构上限”，再定位逐任务 DAG 和共享队列成本
Round 11	cb74bb0	依赖边、提交暂存与线程池重建共同放大固定开销	改为 panel 两阶段屏障，提交深度随参与者扩展，线程池只增不减
Round 13	e09a3ff	每个 MADD 一个 context/submit，任务比队列往返还短	Pass outline range, runtime 按工作量切分，并把预热移出计时区
Round 15	10c91cf	Xeon 上的 50000 FLOP 粒度迁移到 AArch64 后过细	目标平台重扫到 200000 FLOP；建立五段全量测量链，得分 48.19
Round 17–25	ea2be87	无依赖 phase 仍反复经过共享队列；小 tile 缺少 A/B 复用	持久无锁 epoch session + 二维 MADD tiling，得分依次到 51.48、53.14
Round 26–27	ee7d654	MADD 变快后，每 panel 的逐行 TRSM context/submit 成为固定延迟	TRSM range outlining，得分到 53.85
Round 28	9013c03	分支合并后的主线不能直接借用历史成绩	对干净当前源码重建并跑 150 case，确认 $7.386502\times / 53.85$

6.2 瓶颈一：细粒度 DAG 的元数据和共享队列成本

问题。最初把每个 madd 表示为带两个输入依赖的 DAG 节点。理论重叠增加了，但提交线程要为每个节点分配 context、查询 producer key、建立 successor 边并发布到共享队列。在 $n = 1152, b = 16$ 的 profile 中，固定任务数达到 64752，而主线程只执行极少任务；增加核心后，worker 在“等提交线程产生新节点”和“争抢 ready queue”之间振荡。

第一次解决。Round 11 先在可证明无写冲突的位置插入 TRSM/MADD 两阶段 wait，使默认路径不再为每个 MADD 维护依赖边；同时把提交暂存深度与参与者数解耦，避免暂存批次小于等待 worker 数时退化为一任务一次锁，并让线程池跨 case 只增不减。这一步把问题从“复杂 DAG 供给不足”收敛为“无依赖 phase 如何低成本执行”。

最终解决。Round 17 为 $b < 128$ 的无依赖 phase 建立持久 epoch session：提交端一次发布不可变任务表，worker 以静态步长领取，完成时只写入各自 cache line。这样每个 task 不再经过共享队列、mutex 与 futex。以同一 AArch64 全量口径衡量，geomean 从 Round 15 的 $4.370\times$ 提高到 **6.123** $\times$ ，折算总分从 48.19 提高到 **51.48**。

亮点一：让任务表示的成本匹配任务本身

项目没有继续堆叠依赖图功能，而是先证明依赖维护是关键路径，再把可证明安全的 phase 压缩为两个 wait，并把无依赖执行降为 epoch 发布与静态领取。完整 DAG 和 work stealing 仍保留为默认关闭的研究入口，但不再污染已验证主线。

6.3 瓶颈二：任务粒度与目标平台不匹配

问题。单个 MADD 只有约 $2b^3$ FLOP，小 b 时 context 分配、submit 和队列往返可能超过计算本身。Round 10 的隔离探针还表明， $b = 8$ 需要把一整段 k 循环合并后才能摊平固定成本。

编译器方案。Round 13 由 Pass 把连续迭代 outline 成 range task，运行时只接收 $[begin, end]$ 并按工作量切分；任务函数仍逐次调用官方 madd。同时通过 load-time 丢弃并行区完成线程池、worker 栈和 futex 预热，避免首个异步 case 独自承担初始化。

迁移困难。最初的 50000 FLOP 粒度来自 Xeon。换到 AArch64 后，串行 kernel 时间约为 Xeon 的 0.55 倍，相同 FLOP 对应的任务更短，原参数反而过细。Round 15 在目标平台重扫五档预算，把默认值改回 200000 FLOP。受影响的 80 个用例 geomean 提高 5.0%，不受影响的 70 个控制用例只变化 0.2%；全量从 $4.253\times$ 提高到 $4.370\times$ 。

亮点二：Pass 传递语义区间，runtime 决定粒度

粒度不是写死在 IR 中的任务数量，而是由编译器生成可解释的 range，再由通用 runtime 按平台工作量切分。该设计既保留官方 ABI，也让“同一变换、不同平台重新标定”成为可控流程。

6.4 瓶颈三：小 tile 的跨任务访存缺少复用

问题。Round 16 的线程曲线显示，旧实现中 $b = 8$ 从 8 参与者的 $3.379\times$ 下降到 40 参与者的 $2.827\times$ 。这不是并行度不够，而是执行效率随核心数下降。消除共享队列成本后，小 tile 仍很快触及带宽平台，说明只沿 k 粗化的任务会反复流过相同 A/B tile。

编译器方案。Round 20 把 MADD 的三角迭代域改写为二维 (row_group, col_group) range。一个逻辑任务连续完成同一二维 group 内的多个官方 madd 调用，在执行核的 cache 中复用 A/B tile；group width 由 block 数和 tile footprint 共同决定。该改动不修改算子、ABI、运算顺序和

panel barrier。

结果与边界。受影响的 $b < 28$ 组逐用例配对为 $1.3067\times$, $b \geq 28$ 控制组为 $0.9987\times$; 全量 geomean 从 $6.123\times$ 提高到 **$7.006\times$** , 得分到 **53.14**。二维后 $n = 1152, b = 8$ 在 32/40 参与者分别为 $13.45\times/13.55\times$, 表明该桶已从队列瓶颈推进到真实内存带宽平台。

亮点三：在 LLVM Pass 中重排任务域，而不是替换算子

项目以二维任务分组改善局部性，但每个数值更新仍原样进入官方 madd。这是一项可由 IR 语义证明正确的循环/任务域变换，性能收益也由受影响组与控制组的同轮对照确认。

6.5 瓶颈四：每 panel 的 TRSM 固定提交延迟

问题。MADD 路径变快后，提交线程仍要在每个 panel 为每一行 TRSM 分配 context、写参数并 submit。对于小 tile，单次 TRSM 很短，这段串行固定工作进入关键路径；继续调 MADD 粒度已经不能消除它。

编译器方案。Round 26–27 新增 compiler2026_task_trsm_range，每个 panel 只分配一个 context 并提交一次 range；逻辑 index 仍对应一行官方 trsm，grain 固定为 1，TRSM/MADD 与 panel 间的 wait 均保持不变。

机制对照。同轮开/关实验中， $b < 12$ 提高 $1.1400\times$, $12 \leq b < 32$ 提高 $1.0800\times$, 而 $b \geq 32$ 控制组仅 $1.0019\times$ 。全量 geomean 从 $7.005795\times$ 提高到 $7.385822\times$; Round 28 对合并后的当前主线重测为 **$7.386502\times$ / 53.85 分**。

亮点四：持续消除编译器生成的串行固定工作

当数值 kernel 与 MADD 调度都变快后，新的瓶颈转移到 TRSM 的逐迭代提交序列。项目再次使用 range outlining 消除固定延迟，并用“大 tile 控制组不动、小 tile 受影响组提升”证明因果关系。

6.6 困难、负结果与路线收敛

表 6.2: 工程困难与处理原则

困难	风险	处理方式
指标口径曾不一致	suite 总时间比由大用例主导，会把优化优先级带偏	建立逐 case harness，统一使用等权几何平均和评分公式
2 GiB cgroup OOM	150 case 输入 + 输出约 2.46 GB，整套运行 exit 137	按声明内存量切成 5 段，恢复全局 index 后统一评分

困难	风险	处理方式
运行/布局噪声	小于 3% 的总差异可能是漂移, $b = 8$ 还有约 $\pm 10\%$ 布局敏感性	重复测量, 并按机制划分受影响组和控制组
优化破坏数值或发布域	fast phase 曾因任务表部分走快路径、部分走队列而出现 verifier inf	发布域保持完整; 任何候选必须逐 case verifier 与 IR 结构断言同时通过

完整跨 panel DAG、关键路径 rank、两级完成屏障、共享原子 counter、tile packing、静态连续分配、按 phase 宽度缩参与者和 work stealing 原型都经过对照但未超过默认路径, 因此被回退或保持默认关闭。这些负结果不是旁支: 它们逐步排除了“图不够细”“屏障太多”“队列形态必须更复杂”等假设, 使主线最终聚焦在任务元数据、局部性和固定延迟三个可测成本上。

6.7 当前尚未解决的瓶颈

- **小 tile 带宽平台:** $b = 8$ 在 32 核后基本不再扩展; 下一步必须减少必需 C 流量或改变更新复用方式, 单纯增加参与者无效。
- **中等 tile 执行效率:** $12 \leq b < 32$ 与 $32 \leq b < 128$ 分别只达到 Round 15 零开销相位模型上限的约 31.9% 与 63.4%, 且覆盖 100/150 个用例, 是主要分数杠杆。
- **小矩阵每 panel 固定延迟:** TRSM range 已消除一层逐行提交, 但 begin/end、wait、metadata 初始化等固定工作仍会在短矩阵中占主导。
- **到 60 分仍需结构量级:** 当前 $7.3865\times$ 到 60 分所需 $10.667\times$ 仍差全量 $1.444\times$; 小常量、换 barrier 或重新启用 work stealing 都没有这一覆盖面。

第 7 章 后续路线与风险

7.1 优先级路线

1. **减少必需内存流量**: 围绕 C tile 的读写与复用设计新的分组/更新顺序; 现有二维 tiling 已复用 A/B, 但 $b = 8$ 的 32-40 核平台表明 C 流量仍构成硬上限。
2. **继续压缩每 panel 固定工作**: 检查 group decode、context 初始化、range begin/end 和两次 wait 的固定成本, 优先选择能覆盖 $12 \leq b < 128$ 大量用例的 Pass 变换。
3. **形状自适应而非继续拟合单常量**: 联合 $n, b, block_count$ 选择 group width、参与者数和 grain; 候选先做跨矩阵配对, 再进行 150 case 全量验证。
4. **硬件计数器闭环**: 在 AArch64 上记录 bandwidth、cache/TLB、cycles 与同步事件, 验证改动是否真正减少每次官方算子的内存/控制成本。
5. **延后调度结构替换**: 跨 panel DAG、work stealing、barrier 替换已经多轮否认; 只有当前执行效率接近相位模型上限后, 才重新评估 bounded look-ahead 或 NUMA 调度。

7.2 主要风险与缓解

表 7.1: 项目风险

风险	影响	缓解
IR 形态随优化管线变化	Pass 无法识别或误恢复块坐标	版本化 annotation、保守回退、IR 断言
启发式过拟合公开用例	隐藏 case 或目标机退化	分层采样、环境覆盖、目标平台复测、限制常量数量
细粒度并行破坏数值顺序	verifier 失败	只并行无写冲突任务, 阶段 wait, 逐 case verifier
小 tile 在 32 核后带宽饱和	增核不再提高速度, 甚至增加同步和缓存压力	减少 C 流量、硬件计数器、控制参与者
文档与代码默认值漂移	复现实验使用不同阈值	命令显式设置全部关键变量, 并以 CSV 元数据为准

特别需要注意: runtime 当前无环境覆盖时的异步阈值为 8, 而 `submission/scripts/smoke_test.sh` 与 `benchmark.sh` 的脚本默认值仍为 12。Round 28 的逐 case harness 不注入该变量, 使用 runtime 默认值。复现当前基线应使用本章的 per-case 命令; 运行普通 smoke/benchmark 时则应显式写出阈值, 避免把脚本默认与提交默认混用。

第 8 章 复现、构建与证据索引

8.1 构建与 smoke test

在 openEuler/BiSheng 环境中：

```
source /etc/profile.d/bisheng.sh
./submission/scripts/build.sh

# 显式统一异步阈值，避免脚本默认值与 runtime 默认值不同
SPEC_START=91 SPEC_END=96 \
COMPILER2026_DAG_THREADS=40 \
COMPILER2026_ASYNC_MIN_B=8 \
COMPILER2026_ASYNC_MIN_BLOCKS=2 \
./submission/scripts/smoke_test.sh
```

8.2 Round 28 当前基线同口径复现

在 2 GiB 受限的 40 核 AArch64 机器上：

```
LABEL=r28_reproduce PASSES=3 REPEAT=1 \
./scripts/percase_bench_chunked.sh
```

当前正式基线使用 runtime/Pass 默认配置，日志中的 runtime env 应为 <defaults>。输出合并 CSV 位于 docs/benchmark_results/r28_reproduce.csv。任何性能结论都必须同时满足 verifier 全 PASS，并记录 source commit、source dirty 状态、工具链、CPU affinity 和原始 CSV。

8.3 关键证据文件

表 8.1: 报告引用的仓库证据

路径	内容
docs/benchmark_results/r28_head5394c43_cg_baseline.csv	当前 HEAD 的 150 case 合并基线

路径	内容
docs/benchmark_results/r28_head5394c43_baseline.log	源码状态、五段 verifier、IR 断言与评分日志
docs/benchmark_results/r27_cg_trsm_range.csv	TRSM range 后的全量数据
docs/benchmark_results/r26_trsm_range_off.csv	TRSM range 同轮关闭对照
docs/benchmark_results/r26_trsm_range_on.csv	TRSM range 同轮开启实验
docs/benchmark_results/r20_cg_tile_group_default.csv	二维 MADD tiling 全量数据
docs/benchmark_results/r17_cg_fast_phase_default.csv	持久无锁 phase 全量数据
docs/benchmark_results/r15_cg_flops200k.csv	Round 15 AArch64 粒度重标定基线
docs/benchmark_results/r15_cg_range_flops_sweep.csv	五档 range 工作量扫描
docs/benchmark_results/r16_exec_attrib.csv	$b = 8, 12, 32$ 的执行归因
docs/benchmark_results/r22_post_tile_threads.csv	二维 tiling 后的线程扩展与带宽平台证据
docs/roadmap.md	当前基线、已否认路线与下一步约束
docs/engineering_log.md	各轮变更、负结果和经验
docs/technical_scheme_notes.md	赛题规则与评分口径摘录
submission/pass/dag_pass.cpp	当前 Pass 实现
submission/runtime/dag_runtime.cpp	当前 runtime 实现
scripts/score_judge.py	等权逐 case 几何平均与分数计算

8.4 可调参数速查

表 8.2: 主要运行时开关

环境变量	runtime 默认	用途
COMPILER2026_DAG_THREADS	硬件线程数	限制总参与者数
COMPILER2026_ASYNC_MIN_B	8	异步路径最小块大小
COMPILER2026_ASYNC_MIN_B LOCKS	2	异步路径最小块数
COMPILER2026_RANGE_TASK_ FLOPS	200000	range task 目标工作量
COMPILER2026_RANGE_GROUP _WIDTH	自动	二维 MADD group 宽度; 1 关闭分 组
COMPILER2026_TRSM_RANGE	1	Pass 中启用 TRSM range outlining
COMPILER2026_DAG_FAST_PH ASE	$b < 128$ 开启	启用持久无锁 phase 执行
COMPILER2026_DAG_IDLE_SP IN	fast phase 时 20000	phase 间短时自旋预算
COMPILER2026_TASK_BATCH	自动	单次消费批量上限
COMPILER2026_DAG_PARTICI PANT_CAP	形状自适应	显式参与者上限; off 关闭限制
COMPILER2026_DAG_PROFILE	0	输出运行时统计
COMPILER2026_ENABLE_CROS S_PANEL_DAG	0	启用实验性完整 DAG lowering
COMPILER2026_CROSS_PANEL _SYNC_CHOLESKY	0	跨 panel 时保留同步 cholesky
COMPILER2026_DAG_MAX_LIV E	0	实验路径 live-window
COMPILER2026_DAG_CRITICA L_PRIORITY	0	启用 Pass 生成的 rank
COMPILER2026_DAG_PIN_WOR KERS	0	Linux worker affinity 实验

第 9 章 总结

本项目已经完成从官方 baseline IR 到可验证并程序的完整编译链：Pass 负责识别、版本化、任务函数生成与同步插入；通用 runtime 负责线程池、range 切分、批量调度和 profile；测试脚本负责 verifier、IR 结构断言和逐用例评分。当前最有效结果为 AArch64 150/150 PASS、等权几何平均 **7.386502×**、折算总分 **53.85**。

项目最重要的结论并非“依赖越细越好”，而是：依赖表示、任务粒度、队列宽度、初始化成本和评测指标共同决定最终收益。项目沿 Git 历史逐步解决细粒度 DAG 元数据、平台粒度、MADD 局部性和 TRSM 固定提交延迟四个核心瓶颈；每次结构改动都由 verifier、IR 断言、受影响/控制组和全量 CSV 共同确认。下一阶段应优先减少小 tile 的必需内存流量，并继续压缩覆盖中等 tile 的固定工作。